

1- Escolha tipos para um sistema acadêmico

A) Modelagem da relação ALUNO: ALUNO(id, matricula, nome, e_mail, nascimento, coeficiente, bolsista, observacoes).

B) Tipos MySQL para cada atributo:

- id: INT UNSIGNED AUTO_INCREMENT
- matricula: VARCHAR(15)
- nome: VARCHAR(100)
- e_mail: VARCHAR(255)
- nascimento: DATE
- coeficiente: DECIMAL(4,2)
- bolsista: BOOLEAN (ou TINYINT(1))
- observacoes: TEXT

C) Comentários e precisões com justificativas:

- matricula VARCHAR(15): Permite letras/números mantendo zeros à esquerda sem desperdício de espaço.
- nome VARCHAR(100): Acomoda nomes completos extensos sem truncamento.
- e_mail VARCHAR(255): Atende ao padrão máximo recomendado para endereços de e-mail (RFC 5321).
- coeficiente DECIMAL(4,2): Permite notas de \$0.00\$ a \$100.00\$ com precisão exata de duas casas decimais, evitando erros de arredondamento de ponto flutuante.
- observacoes TEXT: Adequado para textos longos de tamanho imprevisível sem limitar o campo a 255 caracteres.

D) Códigos não tratados como números: matricula. Códigos de identificação contêm zeros à esquerda (ex: 00123), podem possuir letras/hífen e não sofrem operações matemáticas (soma, média, etc.).

E) Diferença entre dado ausente, inaplicável e obrigatório:

- Dado obrigatório (NOT NULL): Atributo que precisa ser informado no cadastro (ex: nome, matricula).
- Dado ausente (NULL): Informação aplicável à entidade, mas desconhecida no momento do registro (ex: nascimento não informado).
- Dado inaplicável (NULL): Atributo que não faz sentido para aquela instância específica (ex: motivo do desconto de bolsa para um aluno que não é bolsista).

2- Declare a tabela RESERVA com constraints coerentes

A) Declaração da tabela RESERVA:

SQL

CREATE TABLE RESERVA (

```

id_reserva INT AUTO_INCREMENT,
id_sala INT NOT NULL,
id_responsavel INT NOT NULL,
inicio DATETIME NOT NULL,
fim DATETIME NOT NULL,
situacao VARCHAR(20) DEFAULT 'CONFIRMADA' NOT NULL,
CONSTRAINT pk_reserva PRIMARY KEY (id_reserva),
CONSTRAINT fk_reserva_sala FOREIGN KEY (id_sala) REFERENCES SALA(id_sala),
CONSTRAINT fk_reserva_responsavel FOREIGN KEY (id_responsavel) REFERENCES
PESSOA(id_pessoa),
CONSTRAINT chk_inicio_fim CHECK (fim > inicio),
CONSTRAINT chk_situacao CHECK (situacao IN ('PENDENTE', 'CONFIRMADA',
'CANCELADA', 'CONCLUIDA'))
);

```

B) PK, FKs, NOT NULL e DEFAULT:

- PK: id_reserva identifica de forma única cada reserva.
- FKs: id_sala e id_responsavel garantem integridade referencial com as tabelas de origem.
- NOT NULL: Aplicado a todos os campos vitais para evitar reservas sem responsável, sala ou horário.
- DEFAULT: DEFAULT 'CONFIRMADA' (ou 'PENDENTE') para o atributo situacao.

C) CHECK para fim posterior a início: CONSTRAINT chk_inicio_fim CHECK (fim > inicio).

D) Controle do vocabulário de situacao: CONSTRAINT chk_situacao CHECK (situacao IN ('PENDENTE', 'CONFIRMADA', 'CANCELADA', 'CONCLUIDA')) (ou utilizar o tipo ENUM).

E) Por que UNIQUE(id_sala, inicio) não detecta toda sobreposição temporal: Essa restrição impede apenas reservas que comecem exatamente no mesmo segundo na mesma sala. Ela falha ao detectar sobreposições onde um novo agendamento começa no meio de uma reserva em andamento (ex: Reserva A das 14h às 16h; Reserva B das 15h às 17h possui início diferente, mas entra em conflito de horário).

3- Escolha ON DELETE e ON UPDATE para três vínculos

A) DEPARTAMENTO → EMPREGADO:

- Decisão: ON DELETE SET NULL / ON UPDATE CASCADE.
- Justificativa: O empregado pode permanecer na empresa como funcionário alocado temporariamente sem departamento. A história e a existência do colaborador não dependem da existência do departamento.

B) PROJETO → TAREFA:

- Decisão: ON DELETE CASCADE / ON UPDATE CASCADE.
- Justificativa: A tarefa não possui existência independente (dependência de existência). Excluir um projeto implica o fim de todo o seu ciclo de vida e das atividades a ele vinculadas.

C) AUTOR $\rightarrow$ LIVRO_AUTOR:

- Decisão: ON DELETE RESTRICT / ON UPDATE CASCADE.
- Justificativa: O contrato entre autor e livro é um registro histórico/jurídico. Impedimos a exclusão do autor se houver obras ou contratos associados a ele, garantindo a preservação da memória histórica da publicação.

4- Projete violações que demonstrem a eficácia das constraints

A) Violação de PRIMARY KEY:

SQL

```
-- Inserir duas vezes a mesma PK manual na tabela SALA
INSERT INTO SALA (id_sala, nome) VALUES (1, 'Sala 101');
INSERT INTO SALA (id_sala, nome) VALUES (1, 'Sala 102'); -- Erro: Duplicate entry '1' for
key 'PRIMARY'
```

B) Violação de UNIQUE e CHECK:

SQL

```
-- Violação de UNIQUE (mesmo e-mail cadastrado duas vezes)
INSERT INTO ALUNO (id, e_mail) VALUES (1, 'teste@email.com');
INSERT INTO ALUNO (id, e_mail) VALUES (2, 'teste@email.com'); -- Erro: Duplicate
entry for key 'e_mail'
```

```
-- Violação de CHECK (horário de fim anterior ao início)
```

```
INSERT INTO RESERVA (id_reserva, id_sala, id_responsavel, inicio, fim)
VALUES (10, 1, 5, '2026-10-10 14:00:00', '2026-10-10 13:00:00'); -- Erro: Check
constraint 'chk_inicio_fim' is violated
```

C) Inserção de FK órfã:

SQL

```
-- Tentar reservar para uma sala inexistente (id_sala = 999)
INSERT INTO RESERVA (id_reserva, id_sala, id_responsavel, inicio, fim)
VALUES (11, 999, 1, '2026-10-10 14:00:00', '2026-10-10 15:00:00'); -- Erro: Cannot add
or update a child row: a foreign key constraint fails
```

D) Exclusão bloqueada por RESTRICT:

SQL

```
-- Tentar apagar um Autor que possui registros na tabela associativa LIVRO_AUTOR
```

DELETE FROM AUTOR WHERE id_autor = 1; -- Erro: Cannot delete or update a parent row: a foreign key constraint fails (RESTRICT)

E) Exclusão propagada por CASCADE e estado final esperado:

SQL

-- Executar a exclusão de um projeto

DELETE FROM PROJETO WHERE id_projeto = 50;

- Estado final esperado: O projeto 50 é removido da tabela PROJETO e todas as tarefas associadas a ele (id_projeto = 50) na tabela TAREFA são automaticamente removidas sem deixar registros órfãos.

5- Encontre redundâncias e anomalias em uma relação não normalizada

A) Análise da relação: VENDA(id_venda, data, id_cliente, nome_cliente, id_produto, nome_produto, quantidade).

B) Fatos distintos misturados:

1. O fato da Venda/Transação (data, cliente responsável).
2. O fato do Cliente (identificação e nome).
3. O fato do Produto (identificação e nome do item).
4. O item vendido (quantidade).

C) Demonstração das anomalias:

- Anomalia de Atualização: Se o nome_cliente for alterado, é necessário atualizar dezenas de linhas de vendas antigas; caso contrário, o banco fica em estado inconsistente.
- Anomalia de Inserção: Impossível cadastrar um novo cliente no sistema sem que ele tenha efetuado pelo menos uma compra (ou um novo produto sem que tenha sido vendido).
- Anomalia de Exclusão: Se a única venda de um determinado produto for excluída, o registro de existência do produto é apagado do sistema.

D) Decomposição em tabelas com PKs e FKs:

- CLIENTE(id_cliente [PK], nome_cliente)
- PRODUTO(id_produto [PK], nome_produto, preco_atual)
- VENDA(id_venda [PK], data, id_cliente [FK])
- ITEM_VENDA(id_venda [PK, FK], id_produto [PK, FK], quantidade, preco_unitario_historico)

E) Atributo histórico no item da venda: O atributo preco_unitario (valor negociado no momento da venda) deve permanecer em ITEM_VENDA. Se o preço do produto mudar no cadastro principal (PRODUTO), o histórico do valor cobrado na venda efetuada no passado se mantém inalterado.

6- Implemente o modelo físico de uma clínica

A) Schema e Tabelas (PACIENTE, MEDICO, CONSULTA, PROCEDIMENTO, CONSULTA_PROCEDIMENTO):

SQL

```
CREATE SCHEMA IF NOT EXISTS clinica;  
USE clinica;
```

```
CREATE TABLE PACIENTE (  
    id_paciente INT AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    cpf CHAR(11) NOT NULL,  
    PRIMARY KEY (id_paciente),  
    CONSTRAINT uk_paciente_cpf UNIQUE (cpf)  
);
```

```
CREATE TABLE MEDICO (  
    id_medico INT AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    crm VARCHAR(15) NOT NULL,  
    PRIMARY KEY (id_medico),  
    CONSTRAINT uk_medico_crm UNIQUE (crm)  
);
```

```
CREATE TABLE CONSULTA (  
    id_consulta INT AUTO_INCREMENT,  
    id_paciente INT NOT NULL,  
    id_medico INT NOT NULL,  
    data_hora DATETIME NOT NULL,  
    situacao VARCHAR(15) DEFAULT 'AGENDADA' NOT NULL,  
    PRIMARY KEY (id_consulta),  
    CONSTRAINT fk_consulta_paciente FOREIGN KEY (id_paciente) REFERENCES  
PACIENTE(id_paciente) ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_consulta_medico FOREIGN KEY (id_medico) REFERENCES  
MEDICO(id_medico) ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT chk_consulta_situacao CHECK (situacao IN ('AGENDADA', 'REALIZADA',  
'CANCELADA'))  
);
```

```
CREATE TABLE PROCEDIMENTO (  
    id_procedimento INT AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    valor_base DECIMAL(10,2) NOT NULL,  
    PRIMARY KEY (id_procedimento),  
    CONSTRAINT chk_valor_base CHECK (valor_base >= 0)  
);
```

```
CREATE TABLE CONSULTA_PROCEDIMENTO (
    id_consulta INT NOT NULL,
    id_procedimento INT NOT NULL,
    valor_cobrado DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (id_consulta, id_procedimento),
    CONSTRAINT fk_cp_consulta FOREIGN KEY (id_consulta) REFERENCES
CONSULTA(id_consulta) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_cp_procedimento FOREIGN KEY (id_procedimento) REFERENCES
PROCEDIMENTO(id_procedimento) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_valor_cobrado CHECK (valor_cobrado >= 0)
);
```

B) Escolha de tipos: Identificadores (INT), textos (VARCHAR), instantes de tempo (DATETIME), valores monetários (DECIMAL(10,2)) e estados (VARCHAR com restrição CHECK).

C) Constraints (PK, FK, UNIQUE, NOT NULL, DEFAULT, CHECK): Aplicadas na criação das tabelas acima para assegurar restrições de domínio, unicidade (CPF, CRM) e valores não negativos.

D) Justificativa de ON DELETE / ON UPDATE:

- PACIENTE/MEDICO $\rightarrow$ CONSULTA (RESTRICT): Impede a exclusão acidental de médicos ou pacientes que tenham histórico de consultas no sistema legal/médico.
- CONSULTA $\rightarrow$ CONSULTA_PROCEDIMENTO (CASCADE): Excluir uma consulta cancelada ou removida apaga também a lista de procedimentos atrelados a ela.
- PROCEDIMENTO $\rightarrow$ CONSULTA_PROCEDIMENTO (RESTRICT): Bloqueia a remoção do cadastro de um procedimento se ele já tiver sido cobrado em consultas passadas.

E) Seis testes de violação e resultados previstos:

1. Inserção de CPF duplicado: Gera erro de restrição UNIQUE em PACIENTE.
2. Consulta com data_hora nula: Bloqueado pela restrição NOT NULL.
3. Inserção de consulta com paciente inexistente: Bloqueado por falha de FK (id_paciente).
4. Procedimento com valor negativo: Bloqueado pela constraint CHECK (valor_base >= 0).
5. Exclusão de médico com consultas associadas: Bloqueado pela ação de ON DELETE RESTRICT.
6. Inserção de situação inválida (ex: 'EM_ANDAMENTO'): Bloqueado por violação da constraint CHECK (situacao IN ...).

F) Revisão de redundâncias e dados históricos: O atributo valor_cobrado na tabela associativa CONSULTA_PROCEDIMENTO armazena o preço negociado no

momento do atendimento. Isso impede que reajustes futuros em
PROCEDIMENTO.valor_base alterem retroativamente o histórico financeiro da
clínica.